Taller Semana 4

Librerías de Python para Inteligencia Artificial

Curso: Fundamentos para IA (NRC 94103)

Docente: Cesar Alfonso Bolado Silva

Estudiante(s): [Nombre completo del estudiante]

Fecha de entrega: [dd/mm/aaaa]

Dataset: Netflix TV Shows and Movies — archivo titles.csv (Soeiro, 2022)

Este documento es un ejemplo resuelto del taller, explicado con palabras muy sencillas, ¡como si se lo contáramos a alguien de 10 años! Esta vez investigamos las películas y series de Netflix: cuánto duran, qué calificación tienen y de qué tipo son.

1) Reconocimiento de frameworks de IA en Python

Imagina que quieres construir una casa con bloques de Lego.

- Una librería es como una cajita de piezas ya armadas (por ejemplo, una puerta ya hecha). La usas cuando la necesitas, dentro de lo que estás construyendo.
- Un framework es como el instructivo completo: te dice cómo debe organizarse toda la casa, y tú vas llenando los espacios con tus piezas.

Cuatro librerías/frameworks muy usados en Inteligencia Artificial:

- Scikit-learn: ayuda a la computadora a clasificar cosas o adivinar números (por ejemplo, "¿esta reseña es buena o mala?").
- TensorFlow: un framework de Google para enseñarle a la computadora cosas más difíciles, como reconocer caras en fotos.
- PyTorch: parecido a TensorFlow, muy usado por quienes investigan nuevas formas de inteligencia artificial.
- Hugging Face: se especializa en que la computadora entienda y escriba texto, como un chatbot.

Estas herramientas sirven para: clasificar cosas en grupos, predecir números, reconocer imágenes, entender texto, o agrupar cosas parecidas (esto último se llama clustering).

2) Selección y carga del dataset

Usamos el archivo titles.csv del dataset de Netflix (Soeiro, 2022). Es una ficha por cada película o serie: cuánto dura, de qué año es, qué calificación tiene, etc.

Fuente: https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies

Carga y exploración inicial (head, shape, info)

Cada fila es una película o serie de Netflix. Algunas columnas importantes son:

Columna	Qué significa
title	El nombre de la película o serie
type	Si es MOVIE (película) o SHOW (serie)
release_year	El año en que salió
runtime	Cuánto dura, en minutos
genres	Los géneros (comedia, drama, etc.)
imdb_score	La calificación que le dieron en IMDb
imdb_votes	Cuántas personas la calificaron

Al ejecutar df.shape encontramos 5.850 filas y 15 columnas. df.shape es como contar cuántas fichas hay en una caja de fichas de juego; df.info() nos dice, columna por columna, si guarda texto o números, y cuántos espacios NO están vacíos.

Selección de variables y limpieza

Para no analizar las 15 columnas a la vez, elegimos runtime e imdb_score (numéricas) y type (categórica: MOVIE o SHOW).

Al quedarnos con solo 3 columnas (runtime, imdb_score y type), la tabla reducida muestra 2.290 filas "repetidas", pero no es un error: es normal que muchos títulos distintos compartan, por pura coincidencia, la misma duración, la misma calificación y el mismo tipo. Cada fila sigue siendo un título diferente del archivo original, así que no eliminamos nada por esto. runtime y type no tienen ningún espacio vacío, pero imdb_score sí tiene 482 espacios vacíos: son títulos que todavía no han recibido suficientes votos en IMDb para tener una calificación.

Decidimos imputar (rellenar) esos espacios vacíos con la mediana (6.6) en vez de borrar esas filas, porque son casi 500 títulos: borrarlos nos haría perder muchísima información valiosa sobre duración y tipo de esos títulos. También convertimos type a variable categórica.

Tratamiento de valores atípicos (outliers)

¿Hay películas o series con una duración rarísima (mucho más larga que el resto)? Lo revisamos con más cuidado en la sección de SciPy, usando una regla llamada z-score, y decidimos si los conservamos o no.

3) Implementación con NumPy (cálculo y transformación)

NumPy es una librería súper veloz para hacer cuentas con muchísimos números a la vez, como una calculadora gigante que revisa todos los datos de un solo golpe (sin tener que ir uno por uno con un ciclo for).

Acción 1 — Arreglo: convertimos runtime a un arreglo de NumPy. Un arreglo es como una fila de cajitas numeradas, todas guardando el mismo tipo de dato, lo que hace que las cuentas sean mucho más rápidas que con una lista normal de Python.

Acción 2 — Media, mediana y percentiles: promedio = 76.89 minutos, mediana = 83.0 minutos, percentil 25 = 44.0, percentil 75 = 104.0. El promedio es como repartir en partes iguales; la mediana es "el de en medio"; los percentiles nos dicen en qué posición de la fila está cada título.

Acción 3 — Operación vectorizada (estandarización): restamos la media y dividimos por la desviación estándar a los 5.850 títulos a la vez, sin ningún ciclo for. Esto se llama vectorizar: es mucho más rápido, como usar una fotocopiadora en vez de escribir a mano cada copia. Encontramos 11 títulos con una duración muy larga ($z > 3$).

Acción 4 — np.unique: contamos cuántos títulos hay de cada tipo de una sola vez: 64.00% películas y 36.00% series.

4) Implementación con SciPy (análisis y estadística)

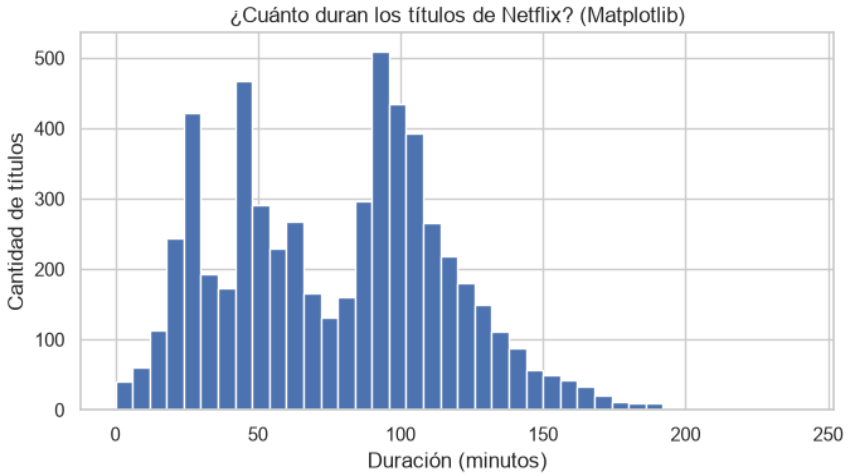
SciPy es una librería que sabe hacer "pruebas" matemáticas más avanzadas: por ejemplo, encontrar los valores más raros de un grupo, o comprobar si dos cosas están relacionadas de verdad (y no solo por casualidad).

Procedimiento 1 — Z-score para atípicos: encontramos 11 títulos con una duración muy fuera de lo común (por ejemplo, Bonnie & Clyde con 240 minutos o Lagaan: Once Upon a Time in India con 224). Al revisar sus nombres, son películas o documentales épicos de verdad. Decidimos conservarlos: no son errores, son casos verdaderos que también forman parte del catálogo de Netflix.

Procedimiento 2 — Correlación de Pearson: entre la duración y la calificación en IMDb obtuvimos r = -0.1439 (p ≈ 1.94e-28). La correlación nos dice si dos cosas se mueven juntas: un número cercano a 0 significa "casi no se relacionan". Aquí encontramos una relación negativa pero débil: los títulos más largos tienden a tener, en promedio, una calificación apenas un poco más baja. El valor p nos dice que sí confiamos en el resultado (no es casualidad).

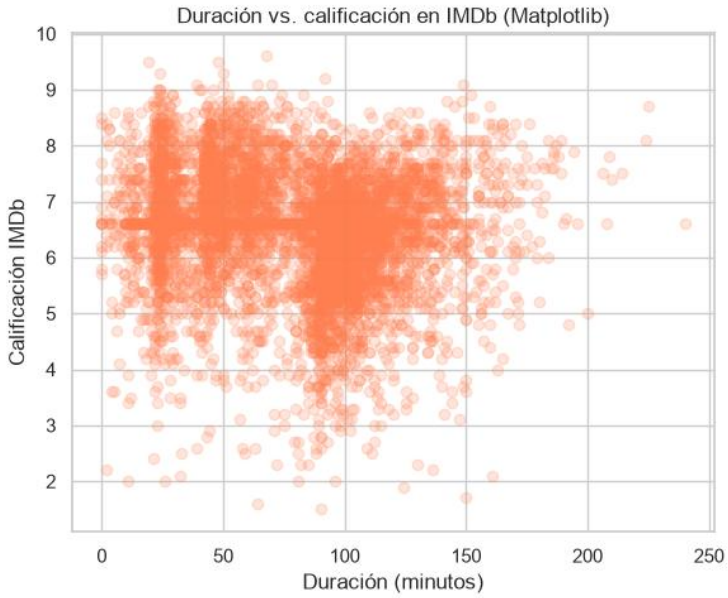
5) Visualización con Matplotlib (mínimo 3 gráficas)

Ahora dibujamos lo que encontramos, para verlo con los ojos y no solo con números. Matplotlib es como una caja de lápices y reglas: nos deja dibujar exactamente lo que queramos, pero paso a paso.



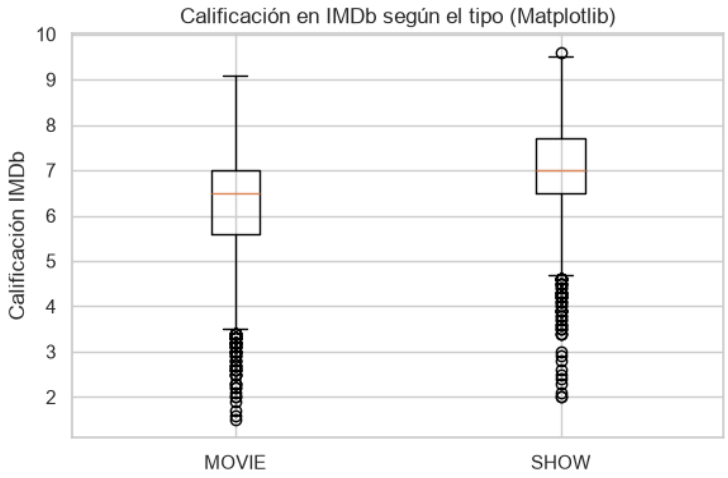
Gráfica 1. Histograma: ¿cuánto duran los títulos de Netflix? (Matplotlib)

Un histograma es como agrupar canicas por tamaño en distintas cajas y ver cuál caja tiene más canicas. Aquí vemos dos "montoncitos": uno cerca de los 30-50 minutos (episodios de series) y otro más grande cerca de los 90-100 minutos (películas).



Gráfica 2. Dispersión: duración vs. calificación en IMDb (Matplotlib)

Un gráfico de dispersión pone un puntito por cada título, según cuánto dura y qué calificación tiene. Aquí la nube de puntos es bastante ancha, lo que coincide con la correlación débil que calculamos con SciPy.

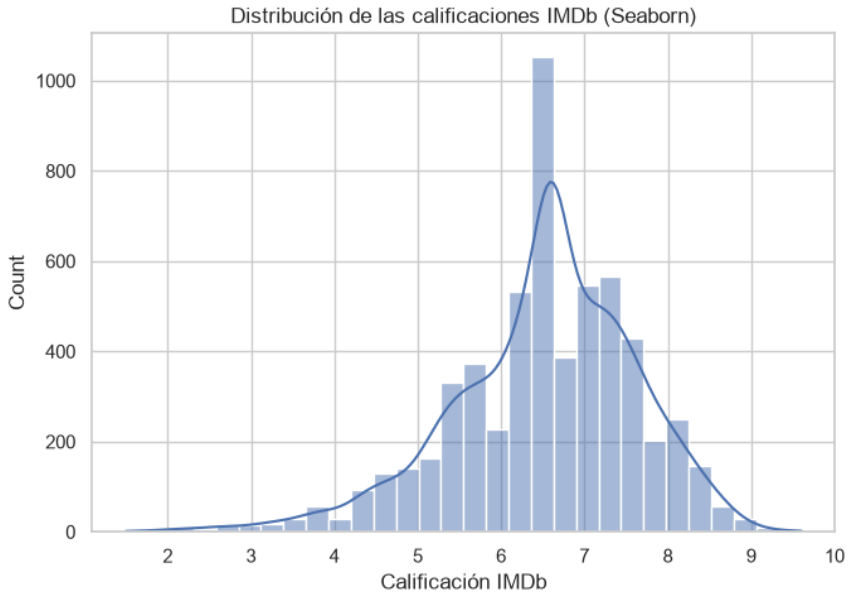


Gráfica 3. Diagrama de caja: calificación en IMDb según el tipo (Matplotlib)

Un diagrama de caja (boxplot) muestra dónde está la mayoría de los datos (la cajita), el valor de en medio (la línea) y los casos raros (los puntitos sueltos). Aquí vemos que las series tienen calificaciones algo más altas que las películas (la mediana es de 7.1 en las series y de 6.4 en las películas); en ambos grupos aparecen casos raros con notas muy bajas o muy altas.

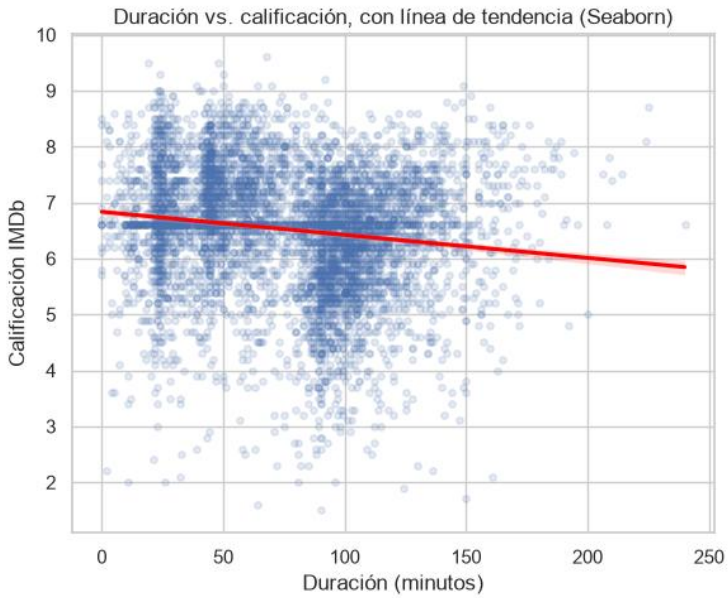
6) Visualización con Seaborn (mínimo 3 gráficas)

Seaborn está construido encima de Matplotlib, pero ya viene con dibujos estadísticos "prearmados" y más bonitos por defecto: es como tener moldes de repostería en vez de cortar la masa a mano.



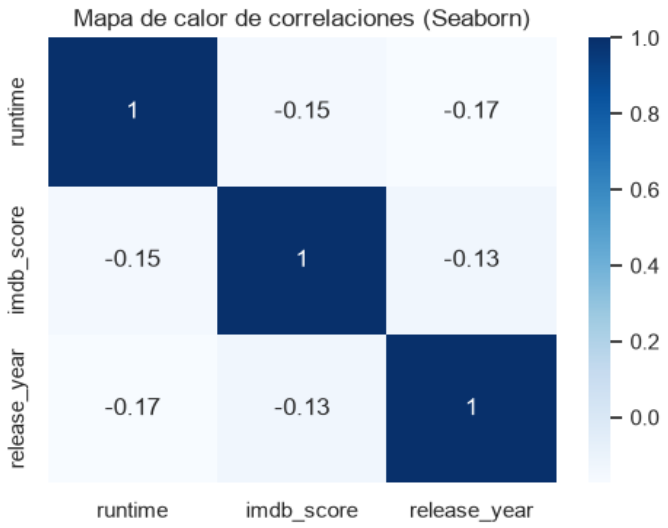
Gráfica 1. Histograma con curva de densidad de las calificaciones IMDb (Seaborn)

sns.histplot con kde=True dibuja el mismo histograma que antes, pero además agrega una línea suave (la curva de densidad) que muestra la forma general de la distribución, sin que tengamos que calcularla nosotros mismos.



Gráfica 2. Dispersión con línea de tendencia (Seaborn)

sns.regplot dibuja los mismos puntos que antes, pero además calcula y dibuja solita la línea de tendencia (la línea roja). Con Matplotlib puro tendríamos que calcular esa línea nosotros mismos con una fórmula aparte.



Gráfica 3. Mapa de calor de correlaciones (Seaborn)

Un mapa de calor (heatmap) es como un cuadro de colores: entre más oscuro (más cerca de 1), más se parecen los movimientos de esas dos variables. Vemos que runtime e imdb_score tienen una relación débil, y que release_year tampoco está muy relacionado con la calificación.

Comparación Matplotlib vs. Seaborn (ejemplo de este taller): para el diagrama de caja de la sección anterior, con Matplotlib tuvimos que separar nosotros mismos los datos en dos listas (películas y series) antes de graficar. Con sns.regplot, en cambio, Seaborn calculó y dibujó la línea de tendencia automáticamente, sin que tuviéramos que hacer ninguna cuenta extra.

7) Pensamiento crítico

¿Cuándo conviene usar Matplotlib y cuándo Seaborn? Matplotlib conviene cuando queremos controlar cada detalle del dibujo, paso a paso (como en nuestro histograma o boxplot). Seaborn conviene cuando queremos un gráfico estadístico bonito y completo en una sola línea, como el regplot que dibujó solo la línea de tendencia entre duración y calificación.

¿Qué ventajas aporta NumPy frente a trabajar solo con Pandas? NumPy hace las cuentas mucho más rápido porque trabaja con arreglos de números "puros" en la memoria de la computadora, sin toda la información extra que carga una tabla de Pandas. Por eso pudimos calcular la media, la mediana y la estandarización de 5.850 títulos de golpe, sin ningún ciclo for.

¿Qué aportó SciPy que no era tan directo con NumPy/Pandas? SciPy nos dio pruebas ya armadas, como el z-score para encontrar valores atípicos y la correlación de Pearson con su valor p. Con NumPy o Pandas solos, tendríamos que calcular esas fórmulas estadísticas nosotros mismos, paso por paso.

Si el objetivo fuera construir un modelo de IA (predicción), ¿qué herramienta sumarían y por qué? Sumaríamos Scikit-learn, porque ya viene con algoritmos listos para, por ejemplo, predecir la calificación de un título según su duración, año y tipo, sin tener que programar esas fórmulas desde cero.

¿Qué limitaciones detectaron en su dataset para aplicar análisis estadístico o visual? Encontramos bastantes valores nulos en imdb_score (títulos sin suficientes votos todavía), que tuvimos que imputar con la mediana. Además, la duración (runtime) mezcla películas y episodios de series, que no siempre son comparables entre sí.

Preguntas del caso

¿Qué fenómeno, proceso o contexto representa el dataset? Representa el catálogo de películas y series disponibles en Netflix: su duración, año de estreno, tipo de contenido y qué tan bien calificadas están por el público.

¿Cuáles son las 5 variables más importantes para describir el caso y por qué?

1. type: distingue si es película o serie, algo fundamental porque el consumo de cada una es muy distinto.

2. runtime: la duración, clave para saber cuánto tiempo pasa un usuario viendo ese título.

3. imdb_score: la calificación, el indicador principal de qué tan buena es la producción según el público.

4. release_year: el año de estreno, útil para ver tendencias del catálogo a través del tiempo.

5. genres: el género, importante para entender qué tipo de historias ofrece Netflix y para sistemas de recomendación.

¿Qué hallazgos se observan en la distribución de variables (categóricas y numéricas)? En la variable categórica type, hay más películas (64%) que series (36%) en el catálogo. En las variables numéricas, runtime tiene una forma con dos "montoncitos" (uno para episodios de series, más cortos, y otro para películas, más largos), y imdb_score se concentra en su mayoría entre 5 y 8 puntos.

¿Qué relación relevante encontraron entre las variables?

- runtime e imdb_score tienen una correlación negativa pero débil (r = -0.14): los títulos más largos tienden a tener, en promedio, una calificación apenas un poco más baja.
- Al revisar el mapa de calor, release_year no muestra una relación fuerte con imdb_score: los títulos más nuevos no son necesariamente mejor ni peor calificados que los más viejos.

¿Qué problemas de calidad de datos detectaron y cómo los abordan? Encontramos muchos valores nulos en imdb_score (títulos sin suficientes votos), que se imputaron con la mediana para no perder esos títulos ni sesgar la calificación promedio. También detectamos títulos con una duración extremadamente larga, que decidimos conservar por ser casos reales (películas y documentales épicos).

Con base en lo observado, ¿qué pregunta de IA o problema predictivo podría plantearse a futuro con este dataset? Se podría plantear un problema de regresión: predecir la calificación de IMDb de un título nuevo a partir de su duración, año y género. También un problema de clasificación para predecir si un contenido nuevo será película o serie según sus características.

Bibliografía

Boschetti, A. y Massaron, L. (2018). Python Data Science Essentials: A Practitioner's Guide Covering Essential Data Science Principles, Tools, and Techniques (pp. 281-331). Packt Publishing Ltd.

Yim, A., Chung, C., & Yu, A. (2018). Matplotlib for Python Developers: Effective Techniques for Data Visualization with Python (2nd ed., pp. 21-54). Packt Publishing, Limited.

Fuentes, A. (2018). Become a Python Data Analyst: Perform Exploratory Data Analysis and Gain Insight into Scientific Computing Using Python (pp. 69-118). Packt Publishing, Limited.

Mehta, H. (2015). Mastering Python Scientific Computing (pp. 163-198). Packt Publishing, Limited.

Soeiro, V. (2022). Netflix TV Shows and Movies [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies